CSC112/256 Fall 2026 Midterm Exam — ANSWER KEY

Student Name: _____ ID: _____

Total Points	

Note: This exam paper should be kept confidential and any dissemination violates copyright.

Q1	Q2	Q3	Q4	Q5
/20	/15	/15	/30	/20

Q1 Multiple-choice questions: enter your answer keys here:

1	2	3	4	5	6	7	8	9	10
A	C	A	B	C	B	C	D	A	D
11	12	13	14	15	16	17	18	19	20
A	C	C	B	B	C	D	D	B	D

Q1. Multiple-choice. (20 pts)
For the following multiple-choice questions, each question has exactly one correct answer key. If multiple choices are correct, choose the option "All of the above". **Fill in the answer keys in the table above. (Answer keys written in the question area will not be counted.)**

1. Which of these is an example of virtualization provided by an OS?
A) A large, private virtual address-space abstraction
B) Direct access to hardware by applications
C) Physical resource duplication
D) Hardware error correction
Answer: A

2. What does a process consist of in an operating system?
A) Only memory and CPU time allocation
B) Only threads of control
C) Address space, threads, and additional system state (e.g., open files, sockets)
D) Only compiled code and libraries
Answer: C

3. Which of the following is NOT considered a core abstraction provided by an OS?
A) Physical hardware duplication for processes
B) Sockets for networks
C) Files for storage devices
D) Threads for processors
Answer: A

4. What does "protection" in an operating system ensure?
A) That users have unrestricted access to hardware resources
B) That processes cannot interfere with each other or the OS itself
C) That all applications run in kernel mode for efficiency
D) That only one application can run at a time on the machine
Answer: B

5. Which of the following is NOT part of a process's memory layout?
A) Code
B) Heap
C) Cache
D) Stack
Answer: C

6. What is the difference between wait() and waitpid()?
A) waitpid() suspends processes, while wait() terminates them.
B) waitpid() allows specifying which child process to wait for, while wait() does not.
C) Both are identical in functionality.
D) wait() waits for all processes, while waitpid() waits only for threads.
Answer: B

7. In user-level threading, what manages thread scheduling?
A) Hardware interrupts
B) System calls
C) User-level thread library
D) Operating system kernel
Answer: C

8. What does the term "process tree" refer to?
A) A hierarchical representation of all running threads within an OS
B) A binary tree used for scheduling algorithms
C) A list of all processes sorted by their priority levels
D) A structure showing parent-child relationships among processes
Answer: D

9. In the Readers/Writers problem, why might writers starve?
A) New readers continuously acquire the lock before writers
B) Writers have higher priority
C) Semaphores are initialized incorrectly
D) Mutex locks are not used
Answer: A

10. What does sem_init(&sem, 0, 3) indicate about the semaphore?
A) Uninitialized semaphore
B) Binary semaphore for mutual exclusion
C) Priority-based semaphore
D) Counting semaphore allowing 3 concurrent accesses
Answer: D

11. What happens when sem_wait() is called on a semaphore with value 0?
A) Blocks until sem_post() is called
B) Causes a segmentation fault
C) Increments the value to +1
D) Returns immediately
Answer: A

12. Which condition variable operation wakes all waiting threads?
A) pthread_cond_init()
B) pthread_cond_wait()
C) pthread_cond_broadcast()
D) pthread_cond_signal()
Answer: C

13. In a Resource-Allocation Graph (RAG), a deadlock is certain if:
A) There is no cycle
B) There is a cycle and each resource has multiple instances
C) There is a cycle and all resources have single instances
D) A thread requests two resources simultaneously
Answer: C

14. In a Resource Allocation Graph (RAG) with a cycle and multi-instance resources:
A) Starvation must occur
B) Deadlock is possible but not certain
C) Deadlock is certain
D) Deadlock is impossible
Answer: B

15. Starvation differs from deadlock because:
A) Starvation cannot occur in priority-based systems
B) Deadlock involves circular waiting
C) Starvation involves circular waiting
D) Starvation always prevents every process from progressing
Answer: B

16. Which is true about the Banker's algorithm?
A) It does not require processes to declare maximum resource needs
B) It uses spooling for printers
C) It requires processes to declare maximum resource needs
D) It prioritizes low-resource threads
Answer: C

17. What characterizes a CPU-bound process?
A) Frequent I/O operations with short CPU bursts
B) Equal distribution of CPU and I/O operations
C) Prioritizes user interaction
D) Long CPU bursts with infrequent I/O waits
Answer: D

18. Which metric is calculated as CompletionTime - ArrivalTime?
A) Throughput
B) Waiting time
C) CPU utilization
D) Response time
Answer: D

19. What is the primary advantage of Shortest Remaining Time First (SRTF)?
A) Maximizes CPU utilization
B) Minimizes average response time
C) Avoids starvation
D) Ensures fairness among all jobs
Answer: B

20. Which scenario benefits most from SRTF?
A) Real-time system
B) Embedded system with fixed tasks
C) Batch processing system
D) Interactive system with mixed job lengths
Answer: D

Sources: Q1 drawn at random from the L1–L5 quizzes, 4 per lecture (option order shuffled; L5 Q4 and Q14 excluded: two keys / needs a figure). Q2(b) adapted from Berkeley CS162 Discussion 1 (fork_loop.c). Q3(c), Q3(d), Q4(c)(2) and Q5(b) adapted from Gate Vidyalay OS practice problems (counting semaphores, deadlock, predicting burst time), with new numbers. Q5(a) is Instance 2 of "New Problem Instances for Q5 Scheduling"; the key above was recomputed (the answer key in that file is not reliable).

Q2 Processes and Threads (15 pts)

For these questions, assume there is no error, i.e., the return value of fork() is never negative. Assume the parent and child may be scheduled in any order unless an explicit synchronization operation imposes an ordering constraint. Assume stdout is connected to a terminal and each newline flushes the displayed line before any relevant fork() or execv() operation. (You need to provide the possible outputs and explain why. You do not need to draw the figures to show the parent child relationships.)

(a) (3 pts) What is the output of this program? If there may be multiple possible outputs, list ALL possible outputs, and explain why. Assume the virtual memory address of variable a in the initial process, &a = 0x7ffc3a10.

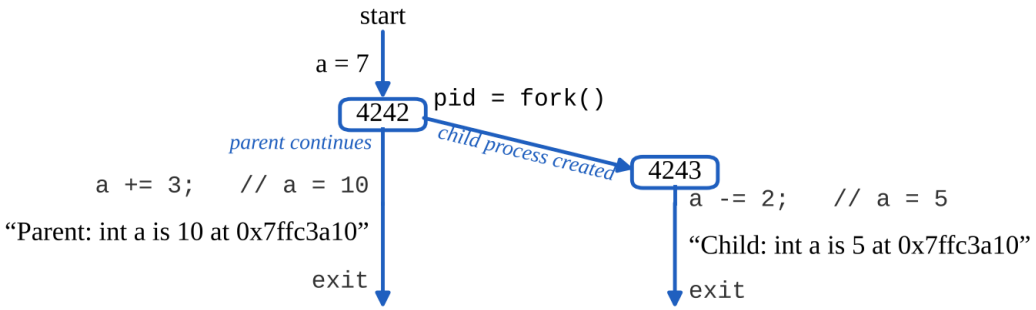
```
1 int main(void) {
2   int a = 7;
3   pid_t fork_ret = fork();
4   if (fork_ret > 0) {
5     a += 3;
6     fprintf(stdout, "Parent: int a is %d at %p\n", a, &a);
7   } else if (fork_ret == 0) {
8     a -= 2;
9     fprintf(stdout, "Child: int a is %d at %p\n", a, &a);
10  } else {
11    printf("Fork error");
12  }
13 }
```

ANS:

Two lines are printed, in either order (arbitrary interleaving of parent and child):

Parent: int a is 10 at 0x7ffc3a10
Child: int a is 5 at 0x7ffc3a10

Explanation: fork() gives the child its own copy of the address space, so the parent computes a = 7+3 = 10 and the child computes a = 7-2 = 5 independently. The child's address space is a copy of the parent's, so &a is the same virtual address 0x7ffc3a10 in both. Both write to the same STDOUT because file descriptors are inherited. Which line appears first depends on the scheduler.



Each process has its own copy of a at the same virtual address 0x7ffc3a10.
No wait(): the two lines can be printed in either order.

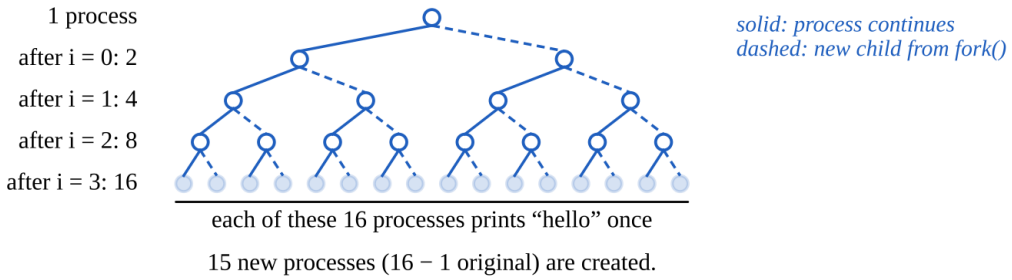
(b) (3 pts) How many new processes (not including the original process) are created when the following program is run, and how many lines of "hello" are printed? Explain.

```
1 int main(void) {
2   for (int i = 0; i < 4; i++)
3     fork();
4   printf("hello\n");
5   return 0;
6 }
```

ANS:

15 new processes (2^4 - 1), and "hello" is printed 16 times (once by each of the 2^4 = 16 processes).

Explanation: a newly forked child continues the loop from where its parent left off, so every fork() doubles the number of processes executing the remaining iterations: 1 → 2 → 4 → 8 → 16 processes after 4 iterations.



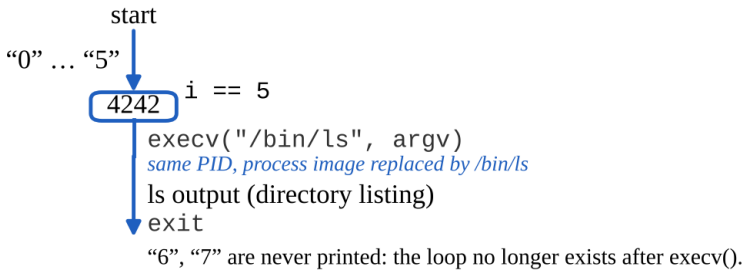
(c) (3 pts) What is the output of this program? If there may be multiple possible outputs, list ALL possible outputs, and explain why.

```
 1 int main(void) {
 2   char** argv = (char**) malloc(3 * sizeof(char*));
 3   argv[0] = "/bin/ls";
 4   argv[1] = ".";
 5   argv[2] = NULL;
 6   for (int i = 0; i < 8; i++) {
 7     printf("%d\n", i);
 8     if (i == 5) {
 9       execv("/bin/ls", argv);
10     }
11   }
12   return 0;
13 }
```

ANS:

0 1 2 3 4 5 (one number per line), followed by the output of ls (the listing of the current directory). Nothing else is printed: 6 and 7 never appear.

Explanation: the loop prints 0 to 5. When i == 5, execv replaces the process image with /bin/ls, which prints the directory listing and exits; the rest of the loop no longer exists.



(d) (3 pts) What is the output of this program? If there may be multiple possible outputs, list ALL possible outputs, and explain why.

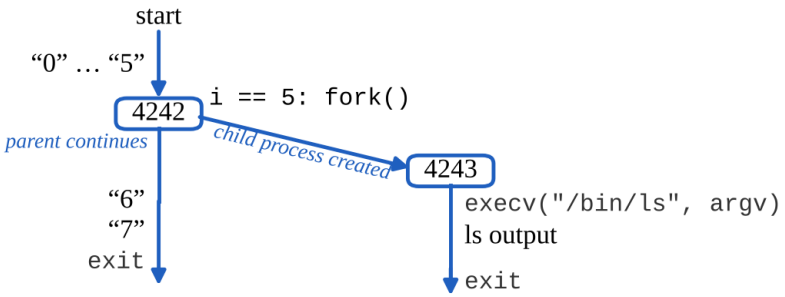
```
1 int main(void) {
2   char** argv = (char**) malloc(3 * sizeof(char*));
3   argv[0] = "/bin/ls";
4   argv[1] = ".";
5   argv[2] = NULL;
6   for (int i = 0; i < 8; i++) {
7     printf("%d\n", i);
8     if (i == 5) {
9       if (fork() == 0)
10        execv("/bin/ls", argv);
11    }
12  }
13  return 0;
14 }
```

ANS:

0 1 2 3 4 5 are printed once (before the fork). Then, with arbitrary interleaving:

- the parent continues the loop and prints 6, 7 (in this order);
- the child (fork() == 0) runs execv and prints the ls output.

So 6 and 7 can appear before, after, or in the middle of the ls listing.



No wait(): "6", "7" and the ls output interleave arbitrarily.

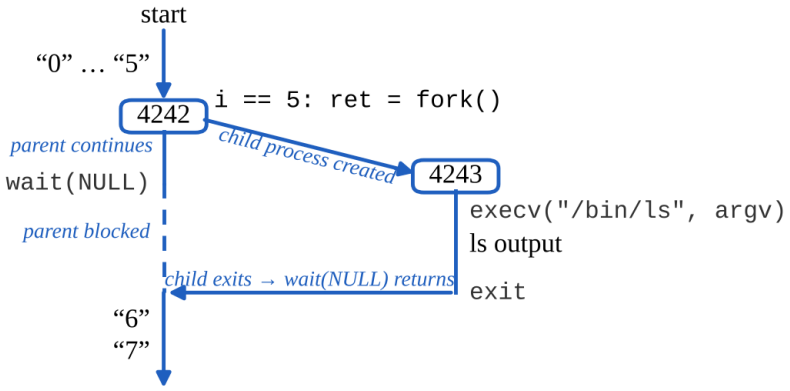
(e) (3 pts) What is the output of this program? If there may be multiple possible outputs, list ALL possible outputs, and explain why.

```
1 int main(void) {
2   char** argv = (char**) malloc(3 * sizeof(char*));
3   argv[0] = "/bin/ls";
4   argv[1] = ".";
5   argv[2] = NULL;
6   for (int i = 0; i < 8; i++) {
7     printf("%d\n", i);
8     if (i == 5) {
9       pid_t ret = fork();
10      if (ret == 0)
11        execv("/bin/ls", argv);
12      if (ret > 0)
13        wait(NULL);
14    }
15  }
16  return 0;
17 }
```

ANS:

Exactly one output, with no interleaving: 0 1 2 3 4 5, then the full ls output, then 6 7.

Explanation: after the fork the child executes ls, while the parent blocks in wait(NULL) until the child terminates. Only then does the parent continue the loop and print 6 and 7.



Output order is fixed: 0 … 5, then the ls output, then 6, 7.

Q3 Synchronization (15 pts)

(a) (4 pts) Consider the following concurrent program, where three threads access a shared variable x within critical sections protected by mutex locks. What are the possible final values of x after all threads finish execution? Explain why.

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; //global mutex
int x = 4; //x is a global shared variable

//Thread T1:
pthread_mutex_lock(&mutex);
x = x + 4;
pthread_mutex_unlock(&mutex);

//Thread T2:
pthread_mutex_lock(&mutex);
x = x - 4;
pthread_mutex_unlock(&mutex);

//Thread T3:
pthread_mutex_lock(&mutex);
x = x / 2;
pthread_mutex_unlock(&mutex);

ANS:

Possible final values: 0, 2, 4.

With the mutex, each update is atomic, so the result only depends on the order of the three critical sections:

T1, T2, T3: 4 → 8 → 4 → 2 T1, T3, T2: 4 → 8 → 4 → 0

T2, T1, T3: 4 → 0 → 4 → 2 T2, T3, T1: 4 → 0 → 0 → 4

T3, T1, T2: 4 → 2 → 6 → 2 T3, T2, T1: 4 → 2 → -2 → 2

(b) (4 pts) Consider the following concurrent program, where three threads access a shared variable x without mutex locks. What are the possible final values of x after all threads finish execution? Explain why.

int x = 4; //x is a global shared variable

//Thread T1:
x = x + 4;

//Thread T2:
x = x - 4;

//Thread T3:
x = x / 2;

ANS:

Possible final values: -2, 0, 2, 4, 6, 8.

Each statement is a load, a compute and a store. An update can be "erased" when another thread loads x before it is stored and stores its own result afterwards (a lost update). So any subset of the three updates can take effect, in any order:

- No update lost: 0, 2, or 4, as in part (a).
- Only T1 takes effect: 4+4 = 8 (maximum). Only T2: 4-4 = 0. Only T3: 4/2 = 2.
- T3 then T1 (T2 lost): 4/2+4 = 6. T3 then T2 (T1 lost): 4/2-4 = -2 (minimum).
- T1 then T3, T2 then T3, T1 then T2, etc. give values already listed (4, 0, 4, ...).

Example for 8: T1 loads 4; T2 loads 4, stores 0; T3 loads 0, stores 0; T1 stores 8.

(c) (3 pts) A counting semaphore S is initialized to 8. Then 13 sem_wait(&S) (P) operations and 9 sem_post(&S) (V) operations are performed on S by different threads, in some order. What is the value of S after all of these operations have completed? Explain.

ANS:

S = 8 - 13 + 9 = 4.

Each sem_wait (P) decrements S by 1 (blocking the caller while S would go below 0) and each sem_post (V) increments S by 1, so once all 22 operations have completed, S = initial value - #P + #V, independent of the order.

(d) (4 pts) A shared variable x is initialized to 10. Four threads W, X, Y and Z run concurrently. W and X each execute x = x + 2, while Y and Z each execute x = x - 3. Each thread calls sem_wait(&s) before reading x and sem_post(&s) after writing x back, where s is a counting semaphore initialized to 2. Each update x = x ± k is performed as: load x into a register, update the register, store it back to x, and a thread can be preempted between any two of these steps. What are the maximum and the minimum possible final values of x after all four threads finish? Explain.

```
sem_t s; sem_init(&s, 0, 2); //counting semaphore, initial value 2
int x = 10; //x is a global shared variable
```

```
//Threads W and X:
sem_wait(&s);
x = x + 2;
sem_post(&s);
```

```
//Threads Y and Z:
sem_wait(&s);
x = x - 3;
sem_post(&s);
```

ANS:

Maximum = 14, minimum = 4.

Because s is initialized to 2, two threads can be inside the read-modify-write at the same time, so one update can overwrite (erase) others.

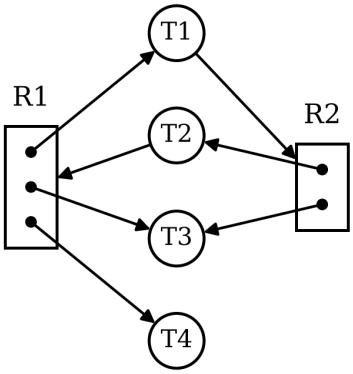
Maximum (both decrements lost): W loads 10 and is preempted; Y loads 10, stores 7, posts; Z loads 7, stores 4, posts; W stores 12 (erasing Y and Z); X loads 12, stores 14. Final x = 14 = 10 + 2 + 2.

Minimum (both increments lost): Y loads 10 and is preempted; W loads 10, stores 12; X loads 12, stores 14; Y stores 7 (erasing W and X); Z loads 7, stores 4. Final x = 4 = 10 - 3 - 3.

(If s were initialized to 1, i.e., a mutex, the only possible result would be 10 + 2 + 2 - 3 - 3 = 8.)

Q4 Deadlocks (30 pts) Consider the following Resource Allocation Graphs (a) and (b), each with 4 threads and 2 resource types. The number of small dots in the box of resource Rj indicates the number of instances of Rj. An arrow from thread Ti to resource Rj indicates that Ti requests 1 instance of Rj; an arrow from Rj to Ti indicates that Ti is holding 1 instance of Rj. Assume each thread's maximum claim is what it currently holds plus what it is requesting (Max = Allocation + Request). For each graph, use Banker's algorithm to determine (1) whether the current state is deadlocked, and (2) whether a safe sequence exists; if one exists, give a safe sequence of thread completions. (You need to give the Max, Allocation, Need matrices, Total and Available vectors, and Available resources after completion of each thread.)

(a) (10 pts) Resource Allocation Graph 1 and corresponding tables.



Max

	R1	R2
T1	1	1
T2	1	1
T3	1	1
T4	1	0

Allocation

	R1	R2
T1	1	0
T2	0	1
T3	1	1
T4	1	0

Need

	R1	R2
T1	0	1
T2	1	0
T3	0	0
T4	0	0

Total

R1	R2
3	2

Available

R1	R2
0	0

Available

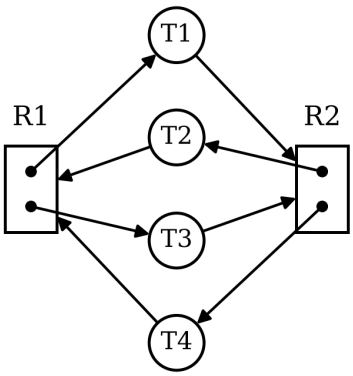
	R1	R2
Init	0	0
T3	1	1
T1	2	1
T2	2	2
T4	3	2

ANS:

(1) Not deadlocked. There is a cycle T1 → R2 → T2 → R1 → T1, but T3 and T4 are not waiting for anything (Need = (0,0)), so they can finish and release their resources.

(2) The state is safe. Safe sequence: T3, T1, T2, T4 (Available after each: (1,1), (2,1), (2,2), (3,2)). Other safe sequences also exist, e.g., T4, T3, T1, T2 or T3, T2, T1, T4.

(b) (10 pts) Resource Allocation Graph 2 and corresponding tables.



Max

	R1	R2
T1	1	1
T2	1	1
T3	1	1
T4	1	1

Allocation

	R1	R2
T1	1	0
T2	0	1
T3	1	0
T4	0	1

Need

	R1	R2
T1	0	1
T2	1	0
T3	0	1
T4	1	0

Total

R1	R2
2	2

Available

R1	R2
0	0

Available

	R1	R2
Init	0	0
Deadlock		

ANS:

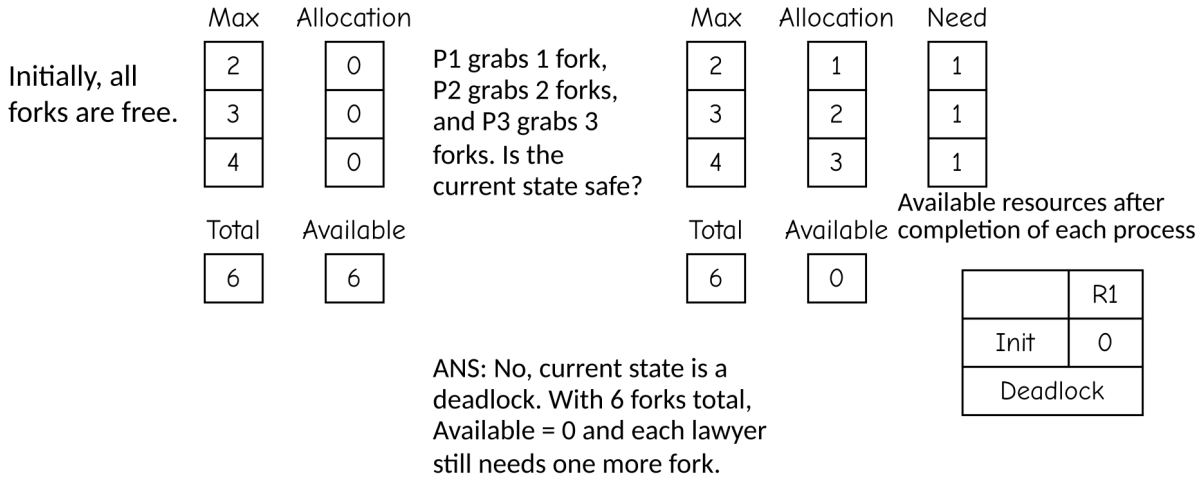
(1) Deadlocked. Available = (0,0) and every thread still needs one more instance: Need(T1) = Need(T3) = (0,1) and Need(T2) = Need(T4) = (1,0). No Need ≤ Available, so no thread can ever proceed (cycles T1 → R2 → T2 → R1 → T1 and T3 → R2 → T4 → R1 → T3, with every instance of R1 and R2 held by a waiting thread).

(2) No safe sequence exists: Banker's algorithm cannot complete even the first thread.

(c) (10 pts) Consider the Dining Lawyers problem. There are 3 lawyers P1 to P3, each with a different number of arms. P1 has 2 arms and needs 2 forks to eat; P2 has 3 arms and needs 3 forks to eat; P3 has 4 arms and needs 4 forks to eat. There is a pile of 6 forks at the center of the table. Each lawyer picks up one fork at a time, and when he gets enough forks, he eats and then puts down all his forks.

(1) (7 pts) Starting from the initial state (no forks held), use Banker's algorithm to determine whether the system can reach an unsafe or deadlocked state. If yes, show such a state and explain why it is deadlocked. If no, explain why every reachable state is safe. (You need to give the Max, Allocation, Need matrices, Total and Available vectors.)

ANS:



Yes. Total = 6, Max = (P1: 2, P2: 3, P3: 4). Initial state: Allocation = (0, 0, 0), Need = Max, Available = 6; this state is safe (e.g., P3, P2, P1 can each get all of their forks in turn).

Reachable deadlocked state: P1 picks up 1 fork, P2 picks up 2 forks, P3 picks up 3 forks. Then Allocation = (1, 2, 3), Need = (1, 1, 1), Available = 6 - 6 = 0.

No lawyer's Need ≤ Available, so the state is unsafe (no safe sequence) and, since every lawyer is waiting for a fork held by another, it is in fact deadlocked. (In general, a deadlock is possible because (2-1) + (3-1) + (4-1) = 6 ≥ 6 forks.)

(2) (3 pts) What is the minimum number of forks in the pile that guarantees that the three lawyers can never be deadlocked, no matter in which order they pick up forks? Explain.

ANS:

7 forks.

In the worst case each lawyer holds one fork less than he needs, i.e., 1 + 2 + 3 = 6 forks, and nobody can eat. With one more fork (6 + 1 = 7), at least one lawyer can always get his last fork, eat, and release all of his forks, after which the others can finish too. With 6 forks the deadlock in part (1) is possible, so 7 is the minimum.

Q5 Scheduling (20 pts)

(a) (16 pts) Consider the set of 2 processes whose arrival time and CPU/IO burst times are given below. For each scheduling algorithm (FCFS, SJF, SRTF, RR, Fixed-Priority (FP)), draw the Gantt chart by filling in the table with the PID that runs in each time slot, and calculate the response time for each process, and the average response time. Assume that context switch overhead is 0. For RR scheduling, the time quantum is 1. For RR, assume that an arriving process (including a process returning from its IO burst) is scheduled to run at the beginning of its arrival time, i.e., it is added to the head of the queue upon arrival. **For FP scheduling, assign P2 (PID 2) higher priority than P1 (PID 1).** In case of a tie, prefer the running process to the newly arrived process, and if everything else is equal, prefer the process with smaller index. If a time slot is idle with no active process executing, then fill in X. (Except for any possible idle time slots at the end of schedule, leave them empty and do not fill in X.)

PID	Arriv. time	CPU Burst	IO Burst	CPU Burst	FCFS Resp. Time	SJF Resp. Time	SRTF Resp. Time	RR Resp. Time	FP Resp. Time
1	0	2	2	5	**10**	**10**	**12**	**12**	**12**
2	1	3	1	2	**11**	**11**	**7**	**8**	**6**
					Avg RT **10.5**	Avg RT **10.5**	Avg RT **9.5**	Avg RT **10**	Avg RT **9**

FCFS	**1**	**1**	**2**	**2**	**2**	**1**	**1**	**1**	**1**	**1**	**2**	**2**		
SJF	**1**	**1**	**2**	**2**	**2**	**1**	**1**	**1**	**1**	**1**	**2**	**2**		
SRTF	**1**	**1**	**2**	**2**	**2**	**1**	**2**	**2**	**1**	**1**	**1**	**1**		
RR	**1**	**2**	**1**	**2**	**2**	**1**	**2**	**1**	**2**	**1**	**1**	**1**		
FP	**1**	**2**	**2**	**2**	**1**	**2**	**2**	**1**	**1**	**1**	**1**	**1**		
Time	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
	↑ P1	↑ P2													

Gantt Chart (↑ marks the arrival time of each process)

ANS:

Response time = completion time - arrival time.

FCFS / SJF (identical): P1 done at 10 (RT 10), P2 done at 12 (RT 11), Avg 10.5. At t=5 P2 is still in IO, so P1 (back from IO at t=4) runs its 5-unit burst non-preemptively.

SRTF: P2 (remaining 2) preempts P1 (remaining 4) when P2 returns from IO at t=6; P1 done at 12 (RT 12), P2 done at 8 (RT 7), Avg 9.5.

RR (q=1): P1 done at 12 (RT 12), P2 done at 9 (RT 8), Avg 10.

FP (P2 > P1): P2 preempts P1 at t=1 and again at t=5; P1 done at 12 (RT 12), P2 done at 7 (RT 6), Avg 9.

(b) (4 pts) The OS predicts the length of the next CPU burst with exponential averaging $\tau_{n+1} = \alpha \cdot t_n + (1 - \alpha) \cdot \tau_n$, where t_n is the actual length of the n-th CPU burst and τ_n is its predicted length. Assume the initial prediction $\tau_0 = 12$, $\alpha = 0.5$, and the actual lengths of the first four CPU bursts t_0, t_1, t_2, t_3 are 8, 4, 10 and 6, respectively. Compute the predicted burst lengths τ_1, τ_2, τ_3 and τ_4.

ANS:

$\tau_1 = 0.5 \times 8 + 0.5 \times 12 = 10$

$\tau_2 = 0.5 \times 4 + 0.5 \times 10 = 7$

$\tau_3 = 0.5 \times 10 + 0.5 \times 7 = 8.5$

$\tau_4 = 0.5 \times 6 + 0.5 \times 8.5 = 7.25$